**`SemaphoreSlim`** is a **.NET** class — it doesn't exist in Angular (which is JavaScript/TypeScript-based).
However, you can **implement similar concurrency control** in Angular/TypeScript to mimic **`SemaphoreSlim`** behavior for limiting the number of concurrent async operations.

Below is a **TypeScript implementation** of a **`SemaphoreSlim`**-like class you can use in Angular:

## `SemaphoreSlim` in TypeScript

```ts
// semaphore-slim.ts
export class SemaphoreSlim {
  private counter: number;
  private readonly maxCount: number;
  private waitingResolvers: Array<() => void> = [];

  constructor(initialCount: number, maxCount?: number) {
    if (initialCount < 0) throw new Error("Initial count must be >= 0");
    this.counter = initialCount;
    this.maxCount = maxCount ?? initialCount;
  }

  /**
   * Waits until the semaphore is available.
   */
  public async wait(): Promise<void> {
    if (this.counter > 0) {
      this.counter--;
```

```
      return;
    }
    // Wait until released
    await new Promise<void>(resolve => this.waitingResolvers.push(resolve));
  }

  /**
   * Releases the semaphore.
   */
  public release(): void {
    if (this.waitingResolvers.length > 0) {
      const resolve = this.waitingResolvers.shift();
      if (resolve) resolve();
    } else if (this.counter < this.maxCount) {
      this.counter++;
    } else {
      throw new Error("Semaphore released too many times");
    }
  }
}
```

## Example Usage in Angular Service

```
import { Injectable } from '@angular/core';
import { SemaphoreSlim } from './semaphore-slim';
```

```
@Injectable({ providedIn: 'root' })
export class ApiService {
  private semaphore = new SemaphoreSlim(2); // Allow max 2 concurrent requests

  async fetchData(url: string): Promise<any> {
    await this.semaphore.wait();
    try {
      const response = await fetch(url);
      return await response.json();
    } finally {
      this.semaphore.release();
    }
  }
}
```

## How It Works

- **`wait()`**: Blocks (asynchronously) until a slot is available.
- **`release()`**: Frees a slot for another task.
- **Use Case in Angular**:
  - Limit concurrent HTTP requests.
  - Prevent overloading APIs.
  - Control access to shared resources.

✅ This approach **mimics .NET's `SemaphoreSlim`** but is fully compatible with Angular/TypeScript async/await patterns.

If you want, I can also give you a **RxJS-based version** that integrates directly with Angular's reactive streams.

Do you want me to prepare that?